

Extending a Pattern

Building classes with commonly named methods creates a consistent pattern that allows us to easily extend the program. For example, to add the ability for our program to include ellipses, we would build an `Ellipse` class that implements the `getArea()`, `clickedInside()`, `draw()`, and `getType()` methods. (The code of the `clickedInside()` method might be mathematically complicated for an ellipse!)

Once we've written the code of the `Ellipse` class, the only change we need to make to the setup code is to add `Ellipse` to the tuple of shape classes to choose from. The code in the main loop that does the checking for clicks, gets the area of the shape, and so on will not need to change at all.

This example demonstrates two important features of polymorphism:

- Polymorphism extends the concept of abstraction discussed in Chapter 8 to a collection of classes. If multiple classes have the same interfaces for their methods, the client programmer can ignore the implementation of those methods in all the classes.
- Polymorphism can make client programming easier. If a client programmer is already familiar with the interfaces provided by one or more classes, then calling the methods of another polymorphic class should as be simple as following the pattern.

pygame Exhibits Polymorphism

All the classes in `pygame` were designed to use polymorphism, and they all implement two common methods. The first is the `handleEvent()` method we first used in Chapter 6, which takes an event object as a parameter. Each class must contain its own code in this method to handle any event that `pygame` may generate. Each time through the main loop, client programs need to call the `handleEvent()` method for every instance of every object instantiated from `pygame`.

Second is the `draw()` method, which draws images to the window. A typical drawing portion of a program that uses `pygame` might look like this: